Evidence-Linked Clinical Trial Criterion Screening with a Compact Fine-Tuned Decoder: KV Caching, Tiled Attention and Speculative Decoding on a CPU

Muhammad Abdullah Masood
Roll No. I230756, Generative AI (Assignment 2)
FAST National University of Computer and Emerging Sciences, Islamabad, Pakistan
Source: https://github.com/AbdullahMasood73/genai-assignment-2

***Abstract*—We build and evaluate a complete, CPU-only system that, for one patient note and one clinical-trial criterion, generates an eligibility label and the IDs of supporting patient sentences. A from-scratch PyTorch implementation of the GPT-NeoX/Pythia forward pass, audited against Hugging Face Transformers, is fine-tuned on the NCBI TrialGPT criterion annotations under the prescribed 37/8/8 patient split. Our first submission (Pythia-14M) collapsed to two labels, cited no evidence and scored below a trivial per-type majority rule; we report this failure and the experiments that followed. The final Pythia-70M model, trained with class-balanced, evidence-weighted answer-only cross-entropy and selected on validation, reaches 63.1% accuracy, present-label macro-F1 0.334 and evidence micro-F1 0.275 on the untouched test patients, against 19.4%, 0.173 and 0.119 before adaptation and 62.6%, 0.272 and 0 for the majority rule. Analysis shows the limits of these gains: inclusion criteria still collapse to "not enough information" and every citation is the opening sentence, so a positional baseline nearly matches the evidence score. We implement and verify a hand-written KV-cached decoder (token-identical to full recomputation; up to 23.71× faster at 1,024 prompt tokens), a tiled online-softmax attention kernel (max error 6.1×10^{-7} versus FP64) and greedy speculative decoding with early-exit and distilled Pythia-14M drafts (token-identical but not reliably faster than cached decoding on a CPU; we measure why). A local browser application exposes criterion-level decisions, verbatim evidence, explicit aggregation rules, report export and live benchmarks. It is an educational prototype, not a clinical tool.**

***Index Terms*—clinical trial matching, eligibility criteria, evidence retrieval, decoder fine-tuning, KV cache, online softmax, speculative decoding**

I. Introduction

Recruiting patients into clinical trials requires checking each inclusion and exclusion criterion against a patient record. Large language models can draft such judgements [1], but a usable screening aid must (i) keep the dataset's full label set, including *not enough information* and *not applicable*, instead of collapsing to yes/no; (ii) cite the exact patient sentences behind a decision, so that absent evidence is never mistaken for proof; and (iii) run on freely available hardware. We therefore ask: *can a small, open, locally executed decoder be adapted to emit a criterion label and supporting sentence IDs, and can its autoregressive inference be accelerated without changing a single output token?*

The dataset provides criterion-level ground truth only. Any patient–trial verdict shown by our application is an aggregation by explicit rules (Section III-G), not a separately labelled outcome. Contributions: (1) a fully reproducible fine-tuning and evaluation pipeline on the prescribed split with leakage checks, a majority baseline and paired before/after comparisons; (2) an honest failure analysis of our first and final models, including a positional evidence baseline; (3) verified implementations of cache-aware decoding, tiled online-softmax attention and greedy speculative decoding with two draft designs; (4) a browser application with live, not pre-recorded, inference comparisons.

II. Related Work

Trial matching. Koopman and Zuccon released the SIGIR patient–trial collection [2] whose patients underlie the TrialGPT annotations. Criteria2Query parses criteria into structured database queries [3]; rule-based parsing is precise for structured fields but brittle for free text. TrialGPT [1] prompts GPT-4 to label each criterion with sentence-level explanations and aggregates criterion scores for ranking; it is accurate but depends on a proprietary API, which the assignment forbids for compulsory parts. We keep TrialGPT's criterion-level task and evidence format but train an open model locally, and we never feed TrialGPT's GPT-4 outputs to our model.

Decoder architectures and adaptation. The Transformer [5] underlies all candidates. Pythia [6] offers one GPT-NeoX architecture [7] at many sizes trained on identical data with Apache-2.0 weights, which makes target/draft pairs with an identical tokenizer available; it uses rotary position embeddings [8] on a fraction of each head. Newer small models (SmolLM2 [9], Qwen2.5 [10]) are stronger per parameter but either lack an in-family smaller draft or need a second architecture implementation and several times our CPU budget (Section III-B). LoRA [11] reduces optimiser state by training low-rank updates; at 19M trainable parameters full updates fit comfortably in 16 GB of RAM, so we fine-tune all blocks directly and leave LoRA as a scaling option. Class-balanced re-weighting [12] counters long-tailed labels such as ours.

Efficient inference. Reusing per-layer keys and values turns each decoding step from $O(t)$ token recomputation into a single-token pass whose attention reads a growing cache [14]; PagedAttention manages this cache in production servers [15]. The online softmax [16] and memory-efficient attention [17]

TABLE I
LABEL DISTRIBUTION PER SPLIT (INCLUSION / EXCLUSION CRITERIA)

Label	Train	Val.	Test
included	88	35	27
not included	13	3	7
excluded	8	2	0
not excluded	292	101	91
NEI (incl./excl.)	109/65	27/26	38/28
not applicable (incl./excl.)	5/23	1/11	3/12
Records with gold evidence	254	90	61
Patients	37	8	8

compute exact attention block by block; FlashAttention [18], [19] fuses this recurrence into IO-aware GPU kernels. Speculative decoding [20], [21] drafts several tokens cheaply and verifies them in one target pass while preserving the target's output distribution (exactly its greedy output in the greedy case). Self-speculative/early-exit drafting uses a prefix of the target's own layers [22], [23]; sequence-level distillation [24] can align a separate small draft with the target. Limitation common to these methods: speed-ups depend on hardware utilisation and acceptance rates, which we measure rather than assume.

III. MATERIALS AND METHODS

A. Dataset, split and leakage control

The NCBI TrialGPT Criterion Annotations [4] contain 1,015 criterion annotations for 53 patients. Each row has a patient note whose sentences are numbered ("0. …"), a criterion type and text, an expert label and expert evidence sentence IDs; GPT-4 predictions, explanations and sentences are also present. Following the protocol, unique patient IDs are sorted, shuffled with `numpy.random.default_rng(42)` and split 37/8/8 into 603/206/206 annotations (Table I). `prepare.py` writes the JSONL splits and a manifest with patient IDs, counts and the source SHA-256. `verify.py` independently re-derives the split from the parquet file, asserts patient disjointness and full coverage, and asserts that every model input contains exactly three fields (note, type, criterion); the three GPT-4 columns are never read into inputs. One training criterion has empty text and is kept with an explicit "[criterion text missing]" marker. Inclusion criteria use {included, not included, not enough information (NEI), not applicable}; exclusion criteria use {excluded, not excluded, NEI, not applicable}. All four are preserved per type. The test split was scored only once, after the configuration was frozen (`results/freeze.json`).

Table I shows the strong imbalance that drives our results: "not excluded" is 48% of training rows, "excluded" has eight training and zero test examples, and only 30–44% of rows have any gold evidence. A per-type majority predictor ("not excluded" for exclusion, "NEI" for inclusion) is therefore a necessary baseline.

B. Model selection and feasibility

All experiments ran on an HP EliteBook 840 G8 (Intel Core i5-1145G7, 4 cores/8 threads, 15.7 GiB RAM, no GPU), FP32 PyTorch 2.8.0 on CPU. Table II compares the candidates.

TABLE II
DECODER CANDIDATES (ALL APACHE-2.0). MEMORY = FP32 WEIGHTS; TRAINING ADDS GRADIENTS AND TWO ADAMW MOMENTS FOR TRAINABLE PARAMETERS.

Model	Params	Layers×d	Ctx	FP32	Decision
Pythia-14M	14.1M	6×128	2k	56 MB	draft; 1st target
Pythia-70M	70.4M	6×512	2k	282 MB	**final target**
Pythia-160M	162M	12×768	2k	650 MB	rejected: ∼4× CPU time
SmolLM2-135M	135M	30×576	8k	540 MB	rejected: new arch., 30 layers
Qwen2.5-0.5B	494M	24×896	32k	2.0 GB	rejected: ∼2 h/epoch

Pythia-14M and -70M share architecture, training data, the 50,277-token byte-level BPE tokenizer (embedding padded to 50,304) and a 2,048-token context, and are Apache-2.0 licensed. We verified that both tokenizer files produce identical token IDs on all 1,015 prompts, which makes 14M a valid speculative draft for 70M.

Pythia-70M has 70.4M parameters, of which 51.5M are the untied input and output embeddings. We (a) freeze the 25.8M-parameter input embedding and (b) replace the 50,304-row output projection with a 107-row *restricted head* initialised from the pretrained unembedding rows of the 107 legal answer symbols (six label codes, a stop symbol ";" and " 0"–" 99"). This leaves 18.97M trainable parameters (Fig. 1 marks the frozen and trained parts): weights, gradients and AdamW moments need about 300 MB, and a full 10-epoch run took 3.6 minutes per epoch on 7 threads. The 14M model was our first target; Section V explains why we moved to 70M. Larger candidates were rejected for CPU training time, not principle.

C. Input/output format and fine-tuning

The prompt is: a fixed instruction listing the codes (A included, B not included, C excluded, D not excluded, E NEI, F not applicable), the criterion type and text, the numbered patient note and "Answer:". The target answer is the label code, the gold sentence IDs in ascending order without duplicates, and the stop symbol, e.g. " A 0 3;". Prompts are 121–453 tokens (train/val/test maxima 453/322/361), so *no note was truncated*; the application rejects prompts above 1,930 tokens rather than silently truncating evidence. At inference a grammar constrains decoding: the first symbol must be one of the four labels valid for the criterion type; afterwards only stop or a strictly larger sentence ID that exists in the note is allowed. Every cited ID is therefore an original sentence ID, and the evidence text is looked up verbatim.

The objective is teacher-forced cross-entropy over answer positions only. Because the first model collapsed to majority labels and empty evidence, we added two re-weightings: the label position is weighted by $(N/(Kn_c))^p$ with training-split class counts n_c, K=6 and p=1, and evidence-ID positions by 5 (stop keeps weight 1). Table V lists the configuration. The checkpoint is selected on validation by the mean of macro-F1 over present labels and evidence micro-F1. A further inference-time parameter, an additive bias b on the stop-symbol logit, was chosen on validation only (Fig. 4); it changes how many sentences are cited but cannot change labels, because stop is not legal at the label position.

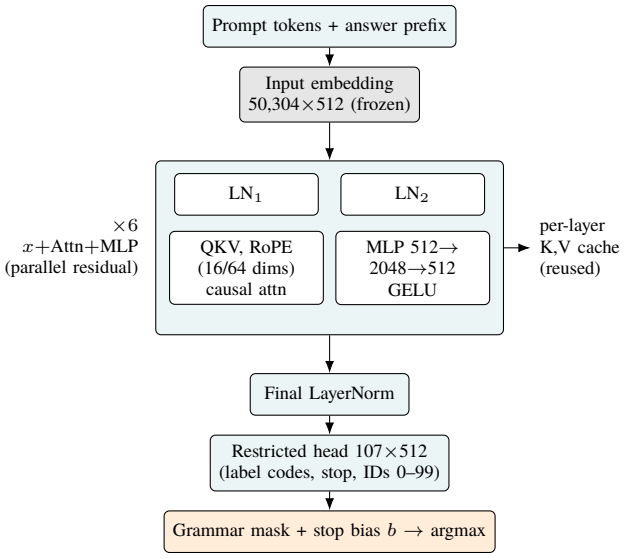


Fig. 1. Implemented Pythia-70M decoder. Grey: frozen; teal: trained (18.97M parameters). Each block caches its rotated keys and values. The early-exit draft reuses the first two blocks, the final norm and the head.

D. KV-cached decoding

Our model code (`model.py`) implements each GPT-NeoX block explicitly: LayerNorm, a fused QKV projection reshaped per head, rotary embedding on the first 25% of each head's dimensions, attention and a parallel MLP residual $x+\text{Attn}(\text{LN}_1 x)+\text{MLP}(\text{LN}_2 x)$. Each block returns its (rotated) keys and values. The *uncached reference* feeds the whole sequence at every step. The *cached path* prefills the prompt once and then feeds only the newest token with the list of per-layer caches. Positional handling: a block receiving n new tokens with a cache of length s assigns absolute positions $s, \ldots, s{+}n{-}1$ to the rotary angles, so cached keys keep the rotation they received when first computed. Masking: on prefill ($s{=}0$) a standard causal mask is used; with a cache, query at absolute position p may attend to keys $0..p$, implemented as the boolean mask `pos[:,None] >= arange(s+n)`, which also supports multi-token chunks used by speculative verification. Cache growth: each step concatenates one key and value per layer, so the FP32 cache costs $2 \cdot L \cdot d \cdot t \cdot 4$ bytes (24 KiB per token for 70M). Numerical differences: the two paths compute the same function with different matrix shapes, so floating-point summation order differs; we measure the logit discrepancy and the smallest top-2 margin to judge whether a greedy flip is possible.

E. Tiled attention with online softmax

The reference kernel materialises $S = QK^\top/\sqrt{d}$, applies the causal mask (aligned to the end of the key sequence so that it also covers cached decoding with $L_q < L_k$), subtracts the row maximum and normalises. The tiled kernel (`inference.py`) processes query tiles of B rows and, for each key tile, updates a running maximum m, normaliser z and numerator a:

$$m' = \max\left(m, \max_j S_{ij}\right), \qquad P = e^{S-m'},$$
$$z' = e^{m-m'} z + \textstyle\sum_j P, \qquad a' = e^{m-m'} a + PV, \quad (1)$$

and returns a/z. Key tiles wholly in the future are skipped; tiles straddling the diagonal use absolute row/column offsets. Only a $B{\times}B$ score tile exists at any time, which is the memory saving FlashAttention exploits on GPUs [18].

Algorithm 1 Greedy speculative decoding (per request)

prefill target and draft on prompt$[:-1]$; $u \leftarrow$ prompt$[-1]$; out$\leftarrow [\,]$
while |out| $<$ limit **do**
  $s \leftarrow$ cached length; $g \leftarrow [\,]$; $x \leftarrow u$
  for $i = 1..k$ **do**
    $g_i \leftarrow \text{argmax}_{\text{legal}}$ Draft(x); $x \leftarrow g_i$; **break if** $g_i =$ stop
  end for
  $\ell \leftarrow$ Target$([u]{+}g)$ ▷ one pass, cache grows by $|g|{+}1$
  for $j = 0..|g|{-}1$ **do**
    $c \leftarrow \text{argmax}_{\text{legal}} \ell_j$; out.append$(c)$
    return if $c =$ stop or budget reached
    if $c \neq g_{j+1}$ **then** trim both caches to $s{+}j{+}1$; $u \leftarrow c$; **break**
    end if
  end for
  if all accepted: feed $g_{|g|}$ to draft; append bonus argmax $\ell_{|g|}$; $u \leftarrow$ bonus
end while

F. Greedy speculative decoding

Algorithm 1 gives our implementation. Both caches always hold the committed prefix *excluding* the pending (last committed, not yet fed) token. The draft proposes up to $k{=}4$ symbols under the same grammar; one target pass over [pending, proposals] scores all positions; the longest agreeing prefix is accepted and the first disagreement is replaced by the target's own symbol (rejection correction). Both caches are then trimmed to the accepted length, so no state from rejected tokens survives. If all proposals are accepted, the target's prediction after the last one is a free bonus token. Hence output equals target greedy decoding by construction, which we also test. We compare two drafts: (i) *early-exit*, the first two adapted 70M blocks with the target's final norm and head (one third of the target's layers, no extra weights); (ii) *Pythia-14M*, distilled on the adapted target's own greedy outputs for the training prompts (sequence-level distillation [24]; teacher-forced agreement on validation rose from 42% to 98%).

G. Application workflow

Fig. 2 summarises the end-to-end workflow. `Start-Screening.cmd` creates a local Python environment on first run, installs pinned dependencies, verifies the pinned base checkpoints by SHA-256 (downloading them from Hugging Face if absent), loads the adapted target and both drafts, and opens http://127.0.0.1:8764. The user pastes a note (numbered lines keep their IDs; free text is split and numbered from 0) and criteria as "inclusion | text" or "exclusion | text" lines, and chooses KV-cached, uncached, speculative or "compare all three" decoding. Each criterion card shows the label, an explanation template grounded in the cited sentences, the verbatim evidence, per-mode latency,

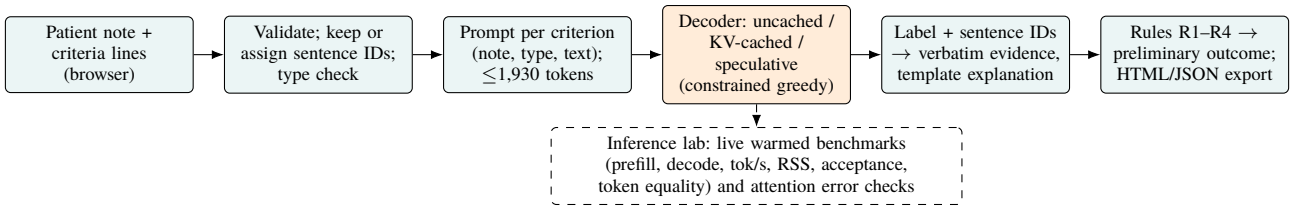


Fig. 2. Application workflow from user input to reviewable, exportable output. Ground truth exists only at criterion level; the overall outcome is a rule-based aggregation.

draft acceptance and token agreement; decisions without citations are flagged *unsupported*. The overall preliminary outcome follows explicit rules: R1 any "not included" or "excluded" → potential ineligibility; R2 otherwise any NEI → undetermined, with the missing items listed; R3 otherwise no conflict found; R4 unsupported decisions are always listed for manual review; R5 an NEI decision that nevertheless cites a sentence is flagged for a consistency check (added after error analysis showed the model often cites the very sentence that resolves an age criterion while answering NEI). Reports export as HTML (readable, printable) or JSON with provenance (model, checkpoint hash, base revision, stop bias, decoding mode, UTC time). An *Inference lab* tab runs fresh, warmed, repeated decoding benchmarks for user-chosen prompt/output lengths and drafts, and live attention checks against FP64; offline logs are shown separately. Invalid input (empty note, malformed criterion, duplicate sentence IDs, oversize prompts, bad benchmark sizes) returns a specific message, and unexpected errors return HTTP 500 without stopping the server.

IV. Experimental Setup and Metrics

Classification. Accuracy, per-class precision/recall/F1 and confusion matrices are reported overall and separately for inclusion and exclusion criteria. Macro-F1 is given two ways: $\mathrm{MF1}_6$ averages all six labels (four per type), counting zero-support classes as 0 as the common protocol's fixed label set implies; $\mathrm{MF1}_p$ averages labels present in the gold split ("excluded" has no test support). **Evidence.** For each record let G and $\hat{G}$ be gold and predicted sentence sets. Micro scores pool $|G \cap \hat{G}|$, $|\hat{G}|$ and $|G|$ over records. Per-record macro scores average precision, recall and F1, scoring 1 when $G = \hat{G} = \emptyset$ and 0 when exactly one is empty; because this rewards "cite nothing" on the 70% of test rows without gold evidence, we also report macro F1 over records with non-empty gold (M^+). **Systems.** All systems use the same rows and the same constrained greedy grammar (no sampling, max 102 symbols): majority baseline; base Pythia-70M; adapted 70M with b=0; adapted 70M with the frozen b=0. **Timing.** Each measurement uses one warm-up call and the median of five synchronous repetitions (CPU kernels are synchronous; a CUDA build would call `synchronize`). Peak memory is the maximum process RSS sampled every 2 ms. Controlled decoding workloads use validation-derived prompts of 32, 256 and 1,024 tokens with 8, 32 and 64 output tokens and the

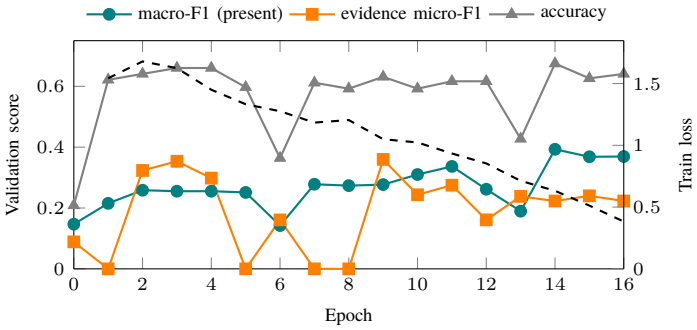


Fig. 3. Validation metrics per epoch (solid, left axis; epoch 0 is the unadapted base) and mean training loss (dashed, right axis). The checkpoint with the best selection score (epoch 9) was frozen.

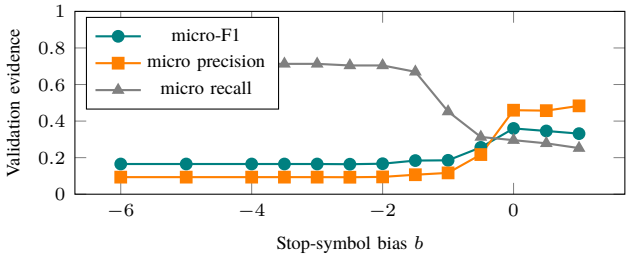


Fig. 4. Validation-only sweep of the additive stop-symbol bias. Lower b makes the decoder cite more sentences, trading precision for recall; $b = 0$ maximises micro-F1 and was frozen before testing.

stop symbol suppressed so all paths emit the same number of tokens; real screening workloads use 24 validation criteria with the full grammar. **Implementation audit.** We compared our forward pass with Hugging Face Transformers' GPT-NeoX (eager attention) on the unadapted checkpoints: all 266 of 266 restricted-vocabulary argmax decisions agree, with maximum absolute logit difference 7.8×10^{-3}.

V. Results and Discussion

A. From a collapsed first model to the final configuration

Table III lists every adaptation run on the validation split, including failures. Our original submission (Pythia-14M, unweighted loss) reached 62.1% validation accuracy, but this is exactly the type-conditioned majority rule: it predicted only "not excluded" and NEI and never cited a sentence. On the test split it scored 0.612 accuracy, below the majority baseline's 0.626 (Table VI). Re-weighting the label loss or the evidence

TABLE III
ALL ADAPTATION RUNS, VALIDATION SPLIT ONLY (BEST EPOCH BY THE SELECTION SCORE)

Configuration	Ep.	Best	Acc.	MF1$_p$	Ev.F1	Sel.
14M, unw. (1st subm.), 2e-4, 4 ep	4	4	0.621	0.219	0.000	0.109
14M, bal. p=0.5, ev+stop ×3, 3e-4	3	1	0.621	0.215	0.000	0.108
14M, unw., ev+stop ×3, 3e-4	3	1	0.621	0.215	0.000	0.108
14M, bal. p=0.5, ev ×5, 3e-4	6	3	0.617	0.213	0.017	0.115
14M, bal. p=0.5, ev ×5, 1e-3	6	5	0.626	0.218	0.000	0.109
70M, bal. p=1, ev ×5, 2e-4, 10 ep	10	10	0.650	0.328	0.250	0.289
70M, bal. p=1, ev ×5, 2e-4, 16 ep	16	9	0.631	0.277	0.360	0.318

"ev+stop" runs used an early weighting bug that also up-weighted the stop symbol; 14M runs were stopped early once collapse was evident. The last row is the frozen model.

TABLE IV
LATENT SIGNAL ON VALIDATION FROM TEACHER-FORCED LOGITS (ROC-AUC; 0.5 = CHANCE)

Model	Mean label	Included	Excl. N/A	Evidence
Pythia-14M base	0.56	0.61	0.29	0.46
Pythia-14M 1st subm.	0.46	0.52	0.44	0.66
Pythia-70M base	0.55	0.56	0.48	0.51
Pythia-70M final	0.53	0.71	0.74	0.69

positions, and raising the learning rate, did not move the 14M model off this collapse. Table IV explains why: the first model's teacher-forced label logits are no better than chance (mean one-vs-rest AUC 0.46), so no decoding trick could recover the labels. Its evidence logits carry weak signal (AUC 0.66), which is why we kept evidence weighting.

Moving to Pythia-70M with fully balanced label weights (p=1) and 5× evidence weighting produced the first non-trivial validation scores. Fig. 3 shows that training is noisy: with eight validation patients, metrics jump between epochs (e.g. epochs 5, 7 and 8 cite no evidence at all before recovering), even as the training loss falls steadily from 1.54 to 0.38. A 16-epoch run beat the 10-epoch run on the pre-declared selection score, and its epoch 9 was frozen. The final model's label AUCs show real but unexploited signal ("included" 0.71, exclusion "not applicable" 0.74; Table IV), a sign of underfitting and miscalibration rather than an absence of signal. The validation stop-bias sweep (Fig. 4) shows a clean precision/recall trade-off but no gain over b=0, so the frozen bias is 0: on this model, citing more sentences only adds wrong citations.

B. Held-out test performance

Table VI is the main paired comparison. On the eight untouched test patients the adapted model is correct on 130 of 206 criteria (63.1%), against 19.4% for the same model before adaptation and 62.6% for the majority rule. Macro-F1 over present labels rises from 0.173 (base) and 0.272 (majority) to 0.334. Evidence micro-F1 rises from 0.119 (base) and 0 (first submission) to 0.275, with precision 0.315 and recall 0.245. These gains are real but small, and two caveats change their interpretation:

1) *The accuracy gain over the majority rule is within noise* (0.631 vs. 0.626, one record). The macro-F1 gain comes from one behaviour: predicting "not applicable" for exclusion criteria (precision 0.75, recall 0.25; Table VIII), mostly pregnancy-type rules for patients to whom they cannot apply.
2) *All 73 predicted citations are sentence 0.* The model has learned *when* to cite the opening sentence (which usually states age and sex and is gold evidence in 38 of the 61 test records with evidence), not *which* sentence supports a criterion. A positional baseline that always cites sentence 0 obtains micro-F1 0.253 and macro F1 over records with evidence 0.508 (Table VI), against the model's 0.275 and 0.340. The model is more precise (0.315 vs. 0.184) but recalls less. This is the most important negative result of the study.

TABLE V
FINE-TUNING CONFIGURATION (PYTHIA-70M TARGET)

Setting	Value
Trainable / total parameters	18,970,112 / 44,725,760
Frozen	input embedding (25.8M)
Objective	answer-only teacher-forced CE
Label weights	balanced, power 1
Evidence-ID weight	5 (stop symbol 1)
Optimiser	AdamW, wd 0.01, clip 1.0
Learning rate	0.0002, 6% warm-up, linear decay
Batch	1 record, accumulate 4
Epochs / selected	16 / 9
Precision / threads / seed	FP32 / 6 CPU / 42

The per-record macro evidence score (Ev.F1$_M$) illustrates the empty-set convention: the majority rule scores 0.704 without retrieving anything, because 145 test records have no gold evidence and an empty prediction counts as perfect there. We therefore treat micro-F1 and M^+ as the evidence metrics of record.

Inclusion vs. exclusion. Tables VII and VIII show the type-specific behaviour. For inclusion criteria (n=75) the greedy decoder predicts NEI every time: accuracy 0.507 equals the NEI share, and "included" (27 cases) is never predicted despite its AUC of 0.71. For exclusion criteria (n=131, accuracy 0.702) the model predicts "not excluded" for 89 of 91 true "not excluded" rows. It also maps all 28 NEI exclusions to "not excluded", the clinically dangerous direction: missing information is read as absence of the exclusion. "Excluded" has no test support, so its row is empty. Table IX confirms that adaptation moved predictions from the base model's arbitrary spread (87 "excluded" predictions) to a prior-matching distribution.

C. Error analysis

There are 122 test records with a wrong label or a non-identical evidence set (76 label errors; 46 records with the correct label but different evidence). We analyse ten distinct error types; IDs are dataset annotation IDs, and full records are in `results/errors.json`.

1) **Correct evidence, unsupported conclusion** (#304, "aged ≥18", 48-year-old male): the model cites the right sentence 0 but outputs NEI. Label and evidence are decoded consistently with the grammar but not with

TABLE VI

PAIRED BEFORE/AFTER COMPARISON ON IDENTICAL VALIDATION AND HELD-OUT TEST ROWS (CONSTRAINED GREEDY DECODING)

Split	System	Acc.	$MF1_6$	$MF1_p$	Acc_{inc}	Acc_{exc}	Ev.P	Ev.R	$Ev.F1_\mu$	$Ev.F1_M$	$Ev.F1^+_M$
Validation	Majority	0.621	0.215	0.215	0.409	0.721	0.000	0.000	0.000	0.563	0.000
Validation	Maj.+lead sent.	0.621	0.215	0.215	0.409	0.721	0.204	0.365	0.262	0.178	0.407
Validation	1st subm. 14M	0.621	0.219	0.219	0.409	0.721	0.000	0.000	0.000	0.563	0.000
Validation	Base 70M	0.209	0.147	0.147	0.530	0.057	0.102	0.078	0.089	0.468	0.071
Validation	Adapted (final)	0.631	0.277	0.277	0.409	0.736	0.459	0.296	0.360	0.602	0.334
Test	Majority	0.626	0.226	0.272	0.507	0.695	0.000	0.000	0.000	0.704	0.000
Test	Maj.+lead sent.	0.626	0.226	0.272	0.507	0.695	0.184	0.404	0.253	0.150	0.508
Test	1st subm. 14M	0.612	0.222	0.266	0.507	0.672	0.000	0.000	0.000	0.704	0.000
Test	Base 70M	0.194	0.144	0.173	0.360	0.099	0.099	0.149	0.119	0.549	0.083
Test	Adapted (final)	0.631	0.279	0.334	0.507	0.702	0.315	0.245	0.275	0.635	0.340

$MF1_6$: macro-F1 over all six labels (zero-support classes count as 0); $MF1_p$: over labels present in the gold set. Ev. = evidence sentence retrieval; μ micro (pooled), M per-record macro where two empty sets score 1, M^+ macro over records with non-empty gold evidence only. Majority = type-conditioned training majority label without evidence; Maj.+lead sent. adds the first note sentence as evidence; 1st subm. = our original Pythia-14M submission re-scored with these metrics.

TABLE VII

ADAPTED MODEL, HELD-OUT TEST, INCLUSION CRITERIA ($n = 75$): PER-CLASS SCORES AND CONFUSION MATRIX (ROWS GOLD, COLUMNS PREDICTED)

Gold	n	P	R	F1	Incl.	Not incl.	NEI	N/A
Incl.	27	0.000	0.000	0.000	0	0	27	0
Not incl.	7	0.000	0.000	0.000	0	0	7	0
NEI	38	0.507	1.000	0.673	0	0	38	0
N/A	3	0.000	0.000	0.000	0	0	3	0

TABLE VIII

ADAPTED MODEL, HELD-OUT TEST, EXCLUSION CRITERIA ($n = 131$): PER-CLASS SCORES AND CONFUSION MATRIX (ROWS GOLD, COLUMNS PREDICTED)

Gold	n	P	R	F1	Excl.	Not excl.	NEI	N/A
Excl.	0	0.000	0.000	0.000	0	0	0	0
Not excl.	91	0.706	0.978	0.820	0	89	1	1
NEI	28	0.000	0.000	0.000	0	28	0	0
N/A	12	0.750	0.250	0.375	0	9	0	3

each other. The application flags such cards for review because the cited sentence visibly contradicts NEI.

2) **Numeric range comparison** (#415, "Age 8–18", 15-year-old girl): gold included; the model outputs NEI with no evidence. Thresholds and ranges are the commonest inclusion failures (also #159, #381, #759).
3) **Missing information read as negation** (#389, "GFR <50%", no renal value in the note): gold NEI, predicted "not excluded". All 28 NEI exclusion rows show this. The application never treats such an uncited decision as proof (rule R4 lists it as unsupported).
4) **Wrong sentence (positional bias)** (#308, informed consent): gold evidence is the consent boilerplate sentence 5; the model cites sentence 0 and predicts NEI.
5) **Multi-sentence reasoning** (#382, "at least 4 ACR criteria for SLE"): the gold decision needs sentences 0–2 (arthralgia, rash, ANA); the model outputs NEI and cites nothing. Our output grammar allows several IDs, but the model never produced more than one.
6) **Minority label never produced** (#836, "no history of wheezing" vs. a wheezing criterion): gold "not included" with sentences 0–1; predicted NEI. "Not included" (7 test cases) is never predicted.
7) **Applicability reasoning** (#750/#751/#758, pregnancy, breast-feeding and "army recruit" for a 5-year-old boy): gold "not applicable"; predicted "not excluded". The model sometimes gets this right (#165, #697) but does not generalise to non-pregnancy rules.
8) **Contradictory annotations** (#322 vs. #165/#697): for a 48-year-old *male*, experts labelled a pregnancy/contraception exclusion "not excluded" (citing sentence 0), whereas pregnancy rules for other patients for whom pregnancy is impossible or irrelevant are labelled "not applicable". The model's "not applicable" for #322 is defensible, so exact-match scoring penalises it.
9) **Evidence present but insufficient** (#193, "patients were moribund"): experts chose NEI yet cited all five clinical sentences; the model predicts "not excluded" without evidence. NEI does not imply empty evidence in this dataset, which complicates both training and the empty-set metric.
10) **Spurious citation under a correct label** (#775, creatinine-clearance exclusion): label correct ("not excluded"), but sentence 0 is cited where experts cite nothing. This costs evidence precision and accounts for many of the 46 label-correct evidence errors.

Missing-information cases (types 3 and 9) matter most for safety. Our mitigations are interface-level: verbatim evidence, an explicit unsupported flag and rule R2/R4 summaries. They are not model-level guarantees.

D. KV caching

Table X reports the controlled decoding benchmark. In all 12 configurations every path produced exactly the uncached

TABLE IX

HELD-OUT TEST PER-LABEL F1 AND PREDICTION COUNTS BEFORE (BASE) AND AFTER ADAPTATION

Label	Support	F1 base	F1 adapt.	#pred base	#pred adapt.
Incl.	27	0.529	0.000	75	0
Not incl.	7	0.000	0.000	0	0
Excl.	0	0.000	0.000	87	0
Not excl.	91	0.000	0.820	0	126
NEI	66	0.224	0.535	41	76
N/A	15	0.111	0.316	3	4

TABLE X
CONTROLLED GREEDY DECODING ON THE ADAPTED 70M TARGET (STOP SUPPRESSED; WARM-UP + MEDIAN OF 5; CPU, 2 THREADS)

In/out	Path	Prefill	Decode	Total ms	Tok/s	×unc.	RSS MiB	KV KiB	Acc./prop.	Equal
32/8	uncached	21.1	169.7	190.8	41.9	1.00	643	–	–	yes
32/8	cached	22.2	43.2	65.3	122.4	2.92	643	960	–	yes
32/8	speculative (early-exit)	–	–	161.5	49.5	1.18	618	–	0/26	yes
32/8	speculative (14M)	–	–	77.9	102.7	2.52	618	–	7/7	yes
256/32	uncached	82.3	2791.5	2873.8	11.1	1.00	678	–	–	yes
256/32	cached	90.2	218.1	308.2	103.8	9.32	677	6912	–	yes
256/32	speculative (early-exit)	–	–	336.1	95.2	8.55	657	–	22/40	yes
256/32	speculative (14M)	–	–	704.0	45.5	4.03	664	–	7/98	yes
1024/64	uncached	387.5	24756.7	25144.2	2.5	1.00	810	–	–	yes
1024/64	cached	395.2	665.4	1060.6	60.3	23.71	806	26112	–	yes
1024/64	speculative (early-exit)	–	–	1124.9	56.9	22.35	799	–	45/74	yes
1024/64	speculative (14M)	–	–	2143.7	29.9	11.85	785	–	12/205	yes

Prefill and decode are timed separately for the two non-speculative paths; speculative time is end-to-end (both prefills included). KV KiB is the analytic FP32 cache size at the end of generation. RSS is sampled process memory (interpreter, all models) and can miss short peaks. Equal: token sequence identical to the uncached reference.

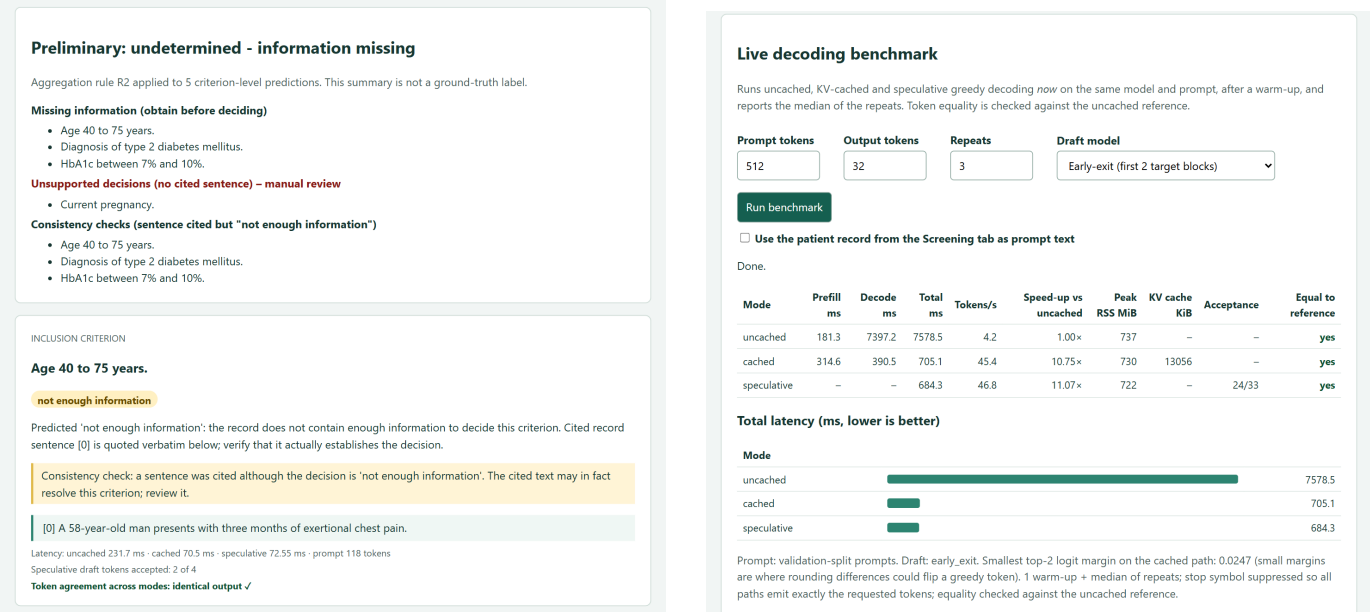


Fig. 5. Left: screening view (final model, compare mode, built-in example) with the applied rule, missing-information, unsupported and consistency-check lists, and a criterion card with verbatim evidence, per-mode latency, draft acceptance and token agreement. It shows the inclusion-side failure discussed in the text: the age sentence is cited, yet the label is NEI, and rule R5 flags it. Right: Inference lab, a fresh warmed benchmark at 512/32 tokens (median of 3) on the same model, with token equality for all three paths.

TABLE XI
REAL SCREENING OUTPUTS (GRAMMAR AND STOP ENABLED) ON 24 VALIDATION CRITERIA: MEAN LATENCY PER CRITERION

Path	n	Mean ms	Out tok.	Accepted	Rate	Equal
uncached	24	184.5	2.33	–	–	24/24
cached	24	88.4	2.33	–	–	24/24
speculative (early-exit)	24	131.7	2.33	24/104	23%	24/24
speculative (14M)	24	106.0	2.33	55/57	96%	24/24

reference tokens (`verify.py` additionally checks 16 prompts). Prefill times are nearly equal between paths, as expected, because both process the full prompt once. The difference lies in decoding: the uncached path re-encodes t tokens at step t, so its decode cost grows quadratically, while the cached path processes one token against a growing cache. The cached speed-up therefore grows with context: 2.92× at 32/8, 9.32× at 256/32 and 23.71× at 1,024/64 tokens. The analytic KV cache is only 24 KiB per token (26 MiB at 1,088 positions), small next to the ≈650–810 MiB process RSS dominated by the interpreter, PyTorch and both models' weights. Measured RSS differences between paths are within sampling noise; on a GPU or with long contexts and batching, cache memory, not compute, becomes the constraint [15]. Cached and full-prefix logits differ by at most 7.3×10^{-4} (chunk test). The smallest top-2 margin observed during cached decoding was 0.017, about 20× larger than that discrepancy, so no greedy flip occurred. Tighter margins could in principle flip a token, which is why the test compares tokens rather than assuming equivalence.

TABLE XII
ATTENTION KERNELS VS. AN FP64 SDPA GROUND TRUTH (4 HEADS, $d_h = 32$, FP32, MEDIAN OF 5)

$L_q \times L_k$	C	Blk	Ref max	Tiled max	Tiled mean	Ref ms	Tiled ms	SDPA ms	Full KiB	Tile KiB
17×17	y	32	5.4e-07	5.4e-07	4.1e-08	0.22	0.39	0.07	5	5
17×17	n	32	5.4e-07	5.4e-07	4.5e-08	0.15	0.25	0.06	5	5
65×65	y	32	4.7e-07	5.0e-07	3.5e-08	0.31	1.43	0.13	66	16
65×65	n	32	3.8e-07	3.8e-07	3.1e-08	0.20	1.43	0.09	66	16
256×256	y	32	6.1e-07	6.1e-07	2.6e-08	1.45	7.35	0.59	1024	16
256×256	n	32	3.5e-07	2.6e-07	1.8e-08	0.52	10.09	0.47	1024	16
512×512	y	32	6.7e-07	6.1e-07	2.1e-08	5.74	25.09	2.05	4096	16
512×512	n	32	5.4e-07	2.6e-07	1.4e-08	2.39	36.24	1.31	4096	16
1×300	y	32	7.0e-08	5.4e-08	1.4e-08	0.18	1.54	0.06	5	0
1×300	n	32	7.0e-08	5.4e-08	1.4e-08	0.13	1.26	0.06	5	0
40×300	y	32	2.4e-07	3.2e-07	1.9e-08	0.35	3.46	0.21	188	16
40×300	n	32	2.8e-07	2.8e-07	1.9e-08	0.24	2.71	0.16	188	16
256×256	y	8	6.1e-07	5.6e-07	2.6e-08	1.36	74.73	0.52	1024	1
256×256	y	16	6.1e-07	5.4e-07	2.5e-08	1.44	20.73	0.55	1024	4
256×256	y	64	6.1e-07	6.1e-07	2.7e-08	1.41	2.66	0.55	1024	64
256×256	y	128	6.1e-07	6.1e-07	3.0e-08	1.27	1.40	0.57	1024	256

Errors are maximum/mean absolute differences from FP64 `scaled_dot_product_attention`. Full/Tile KiB: size of the score matrix materialised by the reference versus the largest single tile of the online-softmax kernel. Rectangular shapes ($L_q < L_k$) model decoding with a KV cache.

E. Tiled attention

Table XII shows that both kernels match an FP64 ground truth on square, rectangular (cached-decoding), causal and non-causal shapes and on block sizes 8–128. The maximum absolute error is 6.1×10^{-7} (reference 6.7×10^{-7}), i.e. FP32 rounding. With logits of magnitude 7,200 a naive exponential overflows, while both max-shifted kernels stay finite and agree to 6×10^{-8}. The tiled kernel stores at most one $B \times B$ score tile (16 KiB at B=32 versus 4 MiB for the full 512×512 matrix), but it is a median 7.9× slower than the reference, which is itself 2.5× slower than PyTorch's fused SDPA. Python loops dispatch many small matrix products, each with fixed overhead and poor cache reuse, whereas fused kernels keep tiles in registers/SRAM and avoid materialising intermediates [18], [19]. At these lengths the full score matrix fits in CPU cache anyway, so the memory saving cannot translate into speed; process RSS does not resolve such small buffers. Larger blocks reduce dispatch overhead: at 256×256 the tiled kernel takes 74.7 ms with B=8 but 1.40 ms with B=128, close to the reference's 1.27 ms. Causal tiling is cheaper than non-causal tiling (7.4 vs. 10.1 ms at 256, 25.1 vs. 36.2 ms at 512) because wholly-future tiles are skipped, the same saving that FlashAttention exploits.

F. Speculative decoding

Speculation produced the target's exact tokens in every benchmark and test, including forced all-accept, all-reject, mid-block stop and budget-limited paths and widths 1–8. In the logged benchmarks it never beat cached greedy decoding on this CPU (Tables X and XI); in one live Inference-lab run at 512/32 tokens the early-exit draft was marginally faster (684 vs. 705 ms at 73% acceptance, Fig. 5), a difference within run-to-run noise. The early-exit draft reached speed-ups versus cached greedy of 0.40, 0.92 and 0.94 with acceptance 0%, 55% and 61%. The distilled 14M draft reached 0.86, 0.43 and 0.50 with acceptance 100%, 7% and 6%. Two measurements explain this. First, *draft cost*: at batch size 1 on CPU a single-token step costs 6.27 ms for the target, 2.15 ms for the early-exit draft and 2.88 ms for the 14M draft. Drafts are only 2–3× cheaper because per-call overhead, not FLOPs, dominates small models. With cost ratio $c \approx 0.35$–0.46 and k=4, speculation needs acceptance well above 60–70% before a round of four draft steps plus one verification beats four target steps. Second, *distribution shift*: the 14M draft was distilled on real screening answers, where it agrees 96% of the time (55/57; Table XI), but the controlled benchmark forces long sequences over a fixed symbol alphabet that the target never produces in practice, and there acceptance falls to 6–7%. The early-exit draft shares the target's lower layers and generalises better to those artificial sequences (55–61%). On real screening outputs (2.33 symbols on average) the 14M draft's high acceptance still loses to cached greedy (106.0 vs. 88.4 ms per criterion), because a second model's prefill and an extra verification pass cannot be amortised over two or three tokens. Speculation pays off for long outputs, with expensive targets, and on hardware where verifying k tokens costs about as much as generating one (GPUs). Our measurements show where those conditions fail.

G. Browser application

Fig. 5 shows the screening view after a comparison run and the Inference lab. All three decoding modes agree on every criterion; latencies and draft acceptance are measured on the user's own request, and the Inference lab reproduces the controlled benchmark live for any prompt length up to 1,024 tokens. During testing the application rejected 14 malformed requests with specific messages (e.g. duplicate sentence IDs, criteria without a type, oversize prompts, non-JSON bodies) and recovered without restarting. We also found and fixed a deployment bug: Python's default `SO_REUSEADDR` on Windows let a second server bind the same port silently, so the launcher could talk to a stale instance. The server now binds exclusively and detects an existing instance through `/health`.

VI. Limitations

Only eight patients (206 correlated criteria) form the test set, so differences of a few points are within sampling noise; "excluded" and several minority classes have tiny or zero support, making per-class scores unstable. The model is small: it learns label priors and lexical cues (age, sex, pregnancy, named diagnoses) far more reliably than numeric thresholds or temporal reasoning, and it still misses most gold evidence. Explanations are templates over the predicted label and the verbatim cited sentences, not generated clinical reasoning; they are faithful to the output but not proof of correctness. The stop bias trades evidence precision for recall and was tuned on only eight validation patients. Benchmarks are CPU-only with Python-level loops; GPU behaviour (where fused kernels and batched verification dominate) would differ. Memory is process RSS, not allocator-level peaks. The application processes text in memory on loopback only and has no authentication; it must not be used with real patient data or for enrollment decisions.

VII. Conclusion

A fully local pipeline, with from-scratch GPT-NeoX inference, fine-tuning, constrained structured decoding and a browser application, can produce criterion-level labels with verbatim, ID-linked evidence on the prescribed TrialGPT split, and its inference optimisations can be verified token for token. The main lessons are methodological. Accuracy alone hid that our first model was a majority rule that cited nothing. A larger decoder with class-balanced, evidence-weighted training improved macro-F1 and evidence F1 on untouched test patients, but the improvements are modest: the model still defaults to "not enough information" for inclusion criteria, reads missing information as "not excluded", and has learned *when* to cite the opening sentence rather than *which* sentence supports a rule. A positional baseline that most papers would omit exposed this. On the systems side, KV caching gave exact speed-ups that grow with context (up to 23.71×); tiled online softmax is exact but slower in Python than a fused kernel; and speculative decoding is exact but needs a draft that is both much cheaper and well aligned with the target, which small models on a CPU do not provide. Future work: a stronger open decoder (e.g. 0.5–1B parameters with LoRA on a free GPU), validation-calibrated per-class priors to exploit the latent signal in Table IV, sentence-level evidence scoring instead of free generation, and patient-level cross-validation for confidence intervals.

Reproducibility

Code, split manifest, configuration, logs, predictions and metrics are at https://github.com/AbdullahMasood73/genai-assignment-2. From the project folder (Python 3.12, `pip install -r requirements.txt`):

1) `python setup_model.py` (download and hash-check bases)
2) `python prepare.py` (split and manifest)
3) `python train.py --base base70 --epochs 16 --lr 2e-4 --schedule linear --class-weight balanced --balance-power 1 --evidence-weight 5 --threads 6 --out experiments/p70_e16`, then copy its `adaptation.pt` to `results/`
4) `python evaluate.py calibrate`; `python distill.py 4 7`
5) `python evaluate.py final`; `python evaluate.py baselines`
6) `python diagnose.py --all`; `python benchmark.py`; `python verify.py`; `python report/tables.py`

The adapted weights (`results/adaptation.pt`, `results/draft14m.pt`) are in the repository and the submission archive. CPU thread count changes floating-point summation order, so retraining reproduces the protocol but not bit-identical epochs (two otherwise identical 10-epoch runs with 6 and 7 threads already differed at epoch 2).

Appendix: AI-use statement

AI assistants were used. OpenAI Codex helped plan and write the first version of the code, run experiments, draft the first report and script the captioned demonstration. Anthropic Claude (Claude Code) later audited that submission, identified the below-baseline result, the lost evidence retrieval and the unaligned draft model, implemented the configuration-driven model code, re-weighted training, calibration, draft distillation, extended tests and benchmarks, the revised application and this revised report. All reused or generated code was executed and tested (`verify.py`); all numbers are produced by the included scripts from logged outputs. The author is responsible for the submission and its defence.

References

[1] Q. Jin *et al.*, "Matching patients to clinical trials with large language models," *Nature Communications*, vol. 15, art. 9074, 2024.
[2] B. Koopman and G. Zuccon, "A test collection for matching patients to clinical trials," in *Proc. SIGIR*, 2016, pp. 669–672.
[3] C. Yuan *et al.*, "Criteria2Query: a natural language interface to clinical databases for cohort definition," *JAMIA*, vol. 26, no. 4, pp. 294–305, 2019.
[4] NCBI, "TrialGPT criterion annotations," Hugging Face dataset, https://huggingface.co/datasets/ncbi/TrialGPT-Criterion-Annotations.
[5] A. Vaswani *et al.*, "Attention is all you need," in *Proc. NeurIPS*, 2017.
[6] S. Biderman *et al.*, "Pythia: A suite for analyzing large language models across training and scaling," in *Proc. ICML*, 2023.
[7] S. Black *et al.*, "GPT-NeoX-20B: An open-source autoregressive language model," in *Proc. BigScience Workshop, ACL*, 2022.
[8] J. Su *et al.*, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, 2024.
[9] L. B. Allal *et al.*, "SmolLM2: When smol goes big," arXiv:2502.02737, 2025.
[10] Qwen Team, "Qwen2.5 technical report," arXiv:2412.15115, 2024.
[11] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.
[12] Y. Cui *et al.*, "Class-balanced loss based on effective number of samples," in *Proc. CVPR*, 2019.
[13] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019.
[14] R. Pope *et al.*, "Efficiently scaling transformer inference," in *Proc. MLSys*, 2023.
[15] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.
[16] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," arXiv:1805.02867, 2018.
[17] M. N. Rabe and C. Staats, "Self-attention does not need $O(n^2)$ memory," arXiv:2112.05682, 2021.
[18] T. Dao *et al.*, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Proc. NeurIPS*, 2022.
[19] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *Proc. ICLR*, 2024.
[20] Y. Leviathan, M. Kalman and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. ICML*, 2023.

[21] C. Chen *et al.*, "Accelerating large language model decoding with speculative sampling," arXiv:2302.01318, 2023.
[22] M. Elhoushi *et al.*, "LayerSkip: Enabling early exit inference and self-speculative decoding," in *Proc. ACL*, 2024.
[23] J. Zhang *et al.*, "Draft & verify: Lossless large language model acceleration via self-speculative decoding," in *Proc. ACL*, 2024.
[24] Y. Kim and A. M. Rush, "Sequence-level knowledge distillation," in *Proc. EMNLP*, 2016.